Calibrated Task-Routed LoRA Banks: Disentangling Routing Mode and Adapter-Birth Initialization for Continual Image Restoration

Anonymous ACCV 2026 Submission

Paper ID #1001

Abstract. Task-routed LoRA banks avoid parameter overwriting in continual image restoration, but prior designs conflate two decisions: which frozen adapters participate in the active route and how each new adapter is initialized. We separate these axes in a controlled design. First, a frozen task-isolated bank makes old-route non-interference a structural property, which persists under enlarged evaluation and longer task sequences. Second, cumulative routing consistently recovers more new-task plasticity than task-only routing by reusing previous adapters as a fixed restoration basis. Third, row-normalizing the LoRA down-projection at birth removes measurable first-step scale leakage across restoration backbones. Enlarged evaluation, however, shows no meaningful final-PSNR advantage over standard initialization; the contribution is therefore a reproducible calibration mechanism rather than a universal performance gain. Finally, a learned-router stress test shows that route prediction, rather than bank interference or adapter initialization, dominates the blind-deployment gap. Together, these results disentangle retention, plasticity, calibration, and routing quality in continual restoration banks.

Keywords: Continual image restoration · LoRA · adapter banks · task routing.

1 Introduction

All-in-one image restoration models handle several degradation types within one system, including rain, snow, haze, and adherent raindrops [8, 18]. In continual deployment these degradations need not arrive together: a model is adapted to one degradation, later updated for another, and earlier restoration skills must remain usable. The setting exposes a retention–plasticity tension. Shared adaptation paths transfer to the newest degradation but let the same parameters overwrite earlier behavior; recent analyses of catastrophic forgetting in all-in-one restoration evaluate exactly this old-task retention after each new task [26].

Parameter-efficient fine-tuning supplies a natural substrate. LoRA freezes the base weight and learns a low-rank residual $\Delta W = (\alpha/r)BA$ [6]. The simplest continual baseline trains one shared LoRA adapter across the task sequence; it keeps high final-task plasticity, but in our three-task chain it loses 9.47 dB on previous tasks. The opposite design is a task-routed adapter bank: one adapter per task, each completed adapter frozen, inference routed by task identity. The non-interference property is then a property of the routing graph rather than of optimization: a completed adapter never changes, so its routed path never changes.

This paper asks how much plasticity the task-routed bank can recover, and **isolates two design axes that prior bank descriptions leave coupled**. The first is the **routing mode** during new-task training. Under task-only routing the active task uses only its newly created adapter; under cumulative routing the active task composes all previous frozen adapters with the current one, reusing earlier updates as a fixed restoration basis while leaving historical routes intact. The second axis is **adapter-birth initialization**. We show that random row-norm variation in the LoRA down-projection leaks into the first effective update scale, and that a one-line row normalization removes this leakage. The two axes are usually designed together in practice – a bank is “cumulative with Kaiming A” or “task-only with PiSSA-initialized A” – so their independent and joint contributions to plasticity are not separately measurable in prior work. We treat them as a 2×2 design and measure row and column marginal effects.

The mechanism is short. Standard LoRA sets A to Kaiming uniform [4] and $B = 0$, so the adapter output is zero at birth. After the first gradient step on B, the rank-wise effective update is

$$\|\Delta W_i\|_F = \frac{\alpha}{r}\,\|B_{\text{new}}[:,i]\|_2\,\|A[i,:]\|_2, \qquad B_{\text{new}} = -\eta\,\nabla_B \mathcal{L}. \tag{1}$$

Random variation in $\|A[i,:]\|_2$ therefore enters the first effective update scale. This is an initialization property, not a trained geometric constraint, and Norm-Bank corrects it by row-normalizing A once at birth while keeping $B = 0$, which preserves the no-op property and removes the rank-wise row-norm variance.

The paper makes four contributions.

1. **Design-space decomposition.** We separate routing mode from adapter-birth initialization on top of a frozen, non-interfering task bank.
2. **Routing controls plasticity.** Cumulative routing gains +0.54 to +0.88 dB across six reduced-sample cells and +0.640 dB on two shared seeds in the 200-image evaluation.
3. **Birth calibration removes scale leakage.** Across 164 layers, row normalization reduces ρ from +0.179 to +0.019 and transfers to Restormer, while the 200-image final-PSNR gap is only +0.016 dB.
4. **Deployment stress test.** A frozen-feature router reaches 84.0% accuracy, with a 1.38–1.41 dB oracle-to-auto gap that exceeds the bank-design difference.

2 Related Work

Continual learning across classification, generation, and reinforcement learning is surveyed in [17]; we focus on the restoration branch that our protocol instantiates.

Restoration. All-in-one restoration handles multiple degradations with one model; PromptIR uses degradation-aware prompts for cross-family restoration [18, 22]. Sequential training changes the problem: new degradations arrive after earlier tasks are learned, and shared updates can erode earlier behavior. This motivates measuring old-task PSNR after each new task rather than only joint multi-task quality [26]. Both PromptIR and our Restormer control build on the vision-transformer architecture [2], which we treat as a fixed backbone rather than as part of the continual-learning design. We adopt a sequential forward protocol and report both old-task PSNR drop and final-task PSNR.

Task-routed LoRA banks. LoRA learns a low-rank residual while freezing the base weight, which fits restoration backbones because it adds few trainable parameters and injects into linear and 1×1 convolutional layers [6]. A task-routed bank keeps one adapter per task, freezes every completed adapter, and routes each input to its task identity. We use the term *routing mode* for the choice of which frozen adapters are composed during new-task training and inference, and *adapter-birth initialization* for the choice of how a newly spawned task adapter is initialized before its first gradient step. Prior bank descriptions couple the two choices – a bank is described as a routing scheme together with an initialization recipe – so their independent and joint contributions to forward plasticity are not separately measurable in prior work. This paper isolates them.

The routing-mode axis: orthogonal-subspace continual learning. Several continual-learning methods constrain the routing between task updates. O-LoRA learns task updates in orthogonal low-rank subspaces for language-model continual learning [24]; InfLoRA constructs an update subspace designed to eliminate interference with old tasks [10]; OA-Adapter pairs dynamic budget allocation with orthogonal-subspace adapter tuning [23]; CL-LoRA combines task-shared and task-specific adapters for rehearsal-free class-incremental learning [3]. Classical baselines isolate parameters or gradients, such as PackNet [15] and Gradient Projection Memory [21]. These works operate on a different layer of the design: they constrain the directions of new updates, whereas the routing-mode axis we study is the composition of frozen updates at forward time. We include row-orthonormal (QR-Bank) and cross-task orthogonal-complement (HOD-Bank) variants as diagnostics; the experiments indicate that unit row scale, not strict orthogonality, is the relevant initialization factor in our main setting.

The adapter-birth axis: initialization of low-rank adaptation. Initialization influences low-rank adaptation. PiSSA initializes from principal singular components of the pretrained weight and shows that spectral initialization shapes optimization [16]. AdaLoRA allocates a heterogeneous rank budget across layers, demonstrating that uniform-rank LoRA is suboptimal when the budget is tight [29]. DoRA decomposes pre-trained weights into magnitude and direction components and shows that reparameterization narrows the gap to full fine-tuning [11]. Norm-Bank operates on a different axis. It does not select pretrained directions, rebalance rank, or split magnitude from direction; it calibrates newly born task adapters by removing row-norm variance in A, targeting the first-step update scale rather than the initialization subspace. PiSSA and AdaLoRA concern what A *is*; Norm-Bank concerns how the rows of A are *scaled*. The two are complementary.

Continual learning foundations. The broader continual-learning literature is grounded in regularization-based consolidation, such as Elastic Weight Consolidation, which penalizes changes to parameters important for past tasks [7], and knowledge-distillation approaches that constrain the current model to imitate the previous one on new data, such as Learning without Forgetting [9]. Gradient-episodic memory [13] constrains new updates so that loss on a small episodic buffer does not increase. Parameter-isolation methods allocate distinct parameter subsets to tasks, including PackNet [15] and Progressive Networks, which dedicates a fresh column to each new task [20]. We focus on image restoration, where retention is measured by per-degradation routed PSNR rather than classification accuracy; evaluation-regime considerations for forgetting measurements are surveyed in [5], and we report both per-task drop and final-task PSNR, and where the parameter-efficient adaptation substrate is LoRA.

3 Preliminaries and Problem Setup

3.1 LoRA Adapters

Let $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ be a frozen weight in a pretrained restoration backbone. A LoRA adapter adds a low-rank residual

$$y = W_0 x + \Delta W x, \qquad \Delta W = \frac{\alpha}{r} BA, \tag{2}$$

with $A \in \mathbb{R}^{r \times d_{\text{in}}}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$, $r \ll \min(d_{\text{in}}, d_{\text{out}})$, and scaling α. Standard LoRA draws A from Kaiming uniform [4] and sets $B = 0$, so the adapter output is zero before training [6].

3.2 Task-Routed LoRA Banks

A task-routed bank keeps one adapter per task. At task t the model holds adapters $\{(A_i, B_i)\}_{i=0}^{t}$ with every completed adapter frozen. Cumulative routing runs the active forward pass through all completed adapters plus the current one,

$$y = W_0 x + \sum_{i=0}^{t} \frac{\alpha}{r} B_i A_i x, \tag{3}$$

while task-only routing uses only the current adapter,

$$y = W_0 x + \frac{\alpha}{r} B_t A_t x. \tag{4}$$

Old-task retention is read through the historical route. After training task 2, task-0 inputs are evaluated on task route 0; because that adapter and the base model are frozen, the task-0 routed computation is unchanged up to numerical effects. Old-task PSNR under task-only routing is a route diagnostic of the preserved historical path: when evaluating task 0 after task 2, the task-0 adapter is the active one and later adapters are excluded from the forward pass, so the measurement verifies that the frozen historical route remains unchanged.

3.3 Continual Restoration Protocol

The forward chain is Rain100L → Snow100K-L → AllWeather Adapt. Task 0 trains on 100 Rain100L samples [27], task 1 on 5,000 Snow100K-L samples [12], and task 2 on 800 Adapt samples from the all-weather benchmark of Li *et al.* [8], which spans rain, snow, haze, and raindrop degradations, with the raindrop family following the protocol of [19]. Rain PSNR after task 0 defines $P^{(0)}_{\text{rain}}$, snow PSNR after task 1 defines $P^{(1)}_{\text{snow}}$, and retention after task 2 is

$$\begin{aligned} \text{rainDrop} &= P^{(0)}_{\text{rain}} - P^{(2)}_{\text{rain}}, \\ \text{snowDrop} &= P^{(1)}_{\text{snow}} - P^{(2)}_{\text{snow}}, \\ \text{meanDrop} &= \tfrac{1}{2}(\text{rainDrop} + \text{snowDrop}). \end{aligned} \tag{5}$$

Lower drop is better, and final-task plasticity is the Adapt PSNR after task 2. The full-method bank comparison uses seeds 42–44, where every method ran; the four-cell routing rerun extends to seeds 42–46. Standard deviations are sample standard deviations.

4 Method: A Two-Axis Decomposition of Task-Routed LoRA Banks

4.1 Routing Modes

The first axis is the active bank route during new-task training and evaluation. **Task-only routing** restricts the current task to its current adapter, which isolates the new adapter and serves as a forward-adaptation control. **Cumulative routing** keeps all previous frozen adapters in the forward path alongside the current adapter; the previous adapters are not updated, but their outputs remain available as a fixed restoration basis. Cumulative routing reuses earlier learned transformations while preserving every historical route, and in our controls it consistently yields higher final-task PSNR than task-only routing.

4.2 Initialization Variants

The second axis is how a newly created task adapter is initialized.

LoRA-Bank initializes each new adapter with Kaiming-uniform A_t and $B_t = 0$, sharing the bank route and freezing rules of the other variants.

Norm-Bank draws A_t from Kaiming uniform and then row-normalizes,

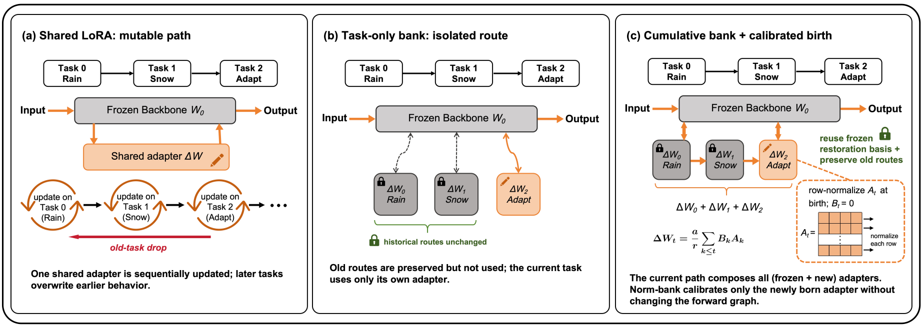

Fig. 1: Routing schematic. **(a)** Shared LoRA sequentially updates a single adapter across all tasks, so later updates overwrite earlier task behavior and old tasks forget (∼9.5 dB drop). **(b)** A task-isolated bank stores one frozen adapter per task; under *task-only* routing, only the active task's adapter is composed at evaluation time. **(c)** Under *cumulative* routing, all completed frozen adapters plus the active adapter are composed, i.e. $\Delta W_t = (\alpha/r) \sum_{k \le t} B_k A_k$. The bank methods preserve old-task routes at reported precision, and **(c)** is the variant that consistently improves forward plasticity on Adapt PSNR.

$$A_t[i,:] \leftarrow \frac{A_t[i,:]}{\|A_t[i,:]\|_2 + \epsilon}, \qquad B_t = 0, \tag{6}$$

which fixes the rank-wise row scale while keeping the adapter a no-op at birth. No per-step retraction, gradient projection, or cross-task projection is applied during training.

QR-Bank replaces row normalization with row-orthonormalization by QR decomposition, giving $A_t A_t^\top = I_r$ when the input dimension is at least r. It tests whether strict row orthogonality beats unit row scale.

HOD-Bank projects the new A_t into the orthogonal complement of previous tasks' A row-spaces and then row-normalizes. It tests whether cross-task row-space separation adds value beyond Norm-Bank. One full-sample seed is missing and the completed seeds are high-variance, so HOD-Bank is reported as a diagnostic.

4.3 Birth-Step Scale-Leakage Diagnostic

Norm-Bank targets an initialization property, so we measure that property directly without full training. We build a LoRA bank with $r = 8$ and $\alpha = 16$, add tasks 0 and 1 with Kaiming initialization, freeze them, and add the task-2 adapter under each variant. A single batch of four Rain100L images yields gradients, and for each LoRA-target layer and rank i we form

$$B_{\text{new}} = -\eta \nabla_B \mathcal{L}, \qquad \|\Delta W_i\|_F = \frac{\alpha}{r} \|B_{\text{new}}[:,i]\|_2 \, \|A[i,:]\|_2, \tag{7}$$

and compute their across-rank Pearson correlation per layer. A positive correlation means random row-norm variation in A has entered the first-step effective update scale. We aggregate the per-layer correlation over the active task-2 adapter only, so frozen Kaiming adapters do not enter the normalized variants' statistics.

4.4 Learned Degradation Router

The bank tables use oracle task routes to isolate the adapter mechanism. Under oracle routing the bank delivers exactly 0.000 dB old-task drop, a property we verify up to $T = 7$ degradations in Supplementary Appendix D; the corresponding forward-latency cost is quantified in Supplementary Appendix C. To probe deployment, we add a learned route predictor: we extract frozen PromptIR latent features, fit a logistic-regression classifier on 50 images per degradation family, and evaluate on 50 held-out images per family. The predicted degradation selects the bank route. The router is deliberately simple; its role is to quantify the oracle-to-auto routing penalty, not to solve blind all-in-one restoration. Router PSNR is measured on the router's own evaluation split and is compared only within its table.

4.5 Implementation

We wrap `nn.Linear` and 1×1 `nn.Conv2d` target layers with a LoRA-bank layer that stores adapters in registered parameter dictionaries. Before each task, existing adapters are frozen and the active adapter is trainable, and the same forward path supports cumulative and task-only routing. We initialize all LoRA down-projections from Kaiming uniform [4] and set the up-projections to zero, following the original LoRA recipe [6]; the PromptIR and Restormer backbones are loaded with their ImageNet-pretrained weights [1]. All trainable LoRA parameters are optimized with AdamW [14] at learning rate 5×10^{-4}. Sanity checks cover frozen-adapter drift, merge equivalence, reload equivalence, and NaN detection.

5 Experiments

5.1 Setup

Backbones. The main experiments use PromptIR with LoRA injected into linear and 1×1 convolutional layers [18]; Restormer is a mechanism and route-preservation control [28].

Training. The full-sample PromptIR chain uses 10 epochs per task, AdamW [14] at learning rate 5×10^{-4}, batch size 4, patch size 128, LoRA rank $r = 8$, and scaling $\alpha = 16$. Reduced-sample controls use 100 Rain100L, 1,000 Snow100K-L, and 400 Adapt images; the full-sample chain uses 100, 5,000, and 800 images, respectively. The reverse-chain control runs Snow100K-L $\rightarrow$ Rain100L $\rightarrow$ Adapt.

Table 1: Initial 20-image evaluation of full-sample continual image restoration on PromptIR. Bank rows use cumulative oracle task routing. Drops are old-task PSNR before task 2 minus old-task routed PSNR after task 2. Mean ± sample std over three seeds.

Method	Adapt PSNR ↑	Rain Drop ↓	Snow Drop ↓	Mean Drop ↓
Shared LoRA	27.421 ± 0.048	+10.605	+8.329	9.467
Norm-Bank	**26.768 ± 0.264**	0.000	0.000	**0.000**
LoRA-Bank	26.467 ± 0.066	0.000	0.000	0.000
QR-Bank	26.584 ± 0.444	0.000	0.000	0.000

Evaluation. Peak signal-to-noise ratio (PSNR) [25] is averaged over five test batches per task in the initial evaluation. To test whether small PSNR differences depend on that 20-image slice, we additionally re-evaluate 14 existing checkpoints from the four Norm/LoRA × cumulative/task-only cells on 200 images, without retraining. Oracle bank rows use the task route of the evaluated degradation; learned-router rows use the predicted route. The oracle and learned-router tables use different evaluation sets and are compared only within each table.

Methods. Shared LoRA uses one adapter across all tasks. LoRA-Bank uses one frozen adapter per task with Kaiming A and $B = 0$. Norm-Bank applies row-normalized A at adapter birth. QR-Bank and HOD-Bank are diagnostic initialization variants.

5.2 Main PromptIR Result: Non-Interference with Recovered Plasticity

Table 1 reports the initial 20-image evaluation of the full-sample PromptIR chain at seeds 42–44. Shared LoRA reaches the highest final Adapt PSNR, 27.421 dB, while losing 9.467 dB on old tasks. Every bank method holds old-task routed performance at 0.000 dB measured drop. On this small evaluation slice, Norm-Bank reaches 26.768 dB, ahead of LoRA-Bank by +0.301 dB under cumulative routing; QR-Bank sits within seed noise. HOD-Bank reaches 26.791 ± 0.636 dB at 0.000 mean drop, but only two of its seeds completed, so it stays a diagnostic. Section 5.3 reports the larger 200-image control used for the final effect-size interpretation.

5.3 Five-Seed Full-Sample Routing Control: The 2×2 Design

Table 2 lays out the initial 20-image 2×2 evaluation on PromptIR at full sample, seeds 42–46, with five seeds per cell. Rows fix the routing mode and vary the initialization; columns fix the initialization and vary the routing mode. Across all 20 bank runs, rain and snow retention drops are 0.000 dB at reported precision. Table 3 then enlarges the evaluation to 200 images for 14 available checkpoints without retraining.

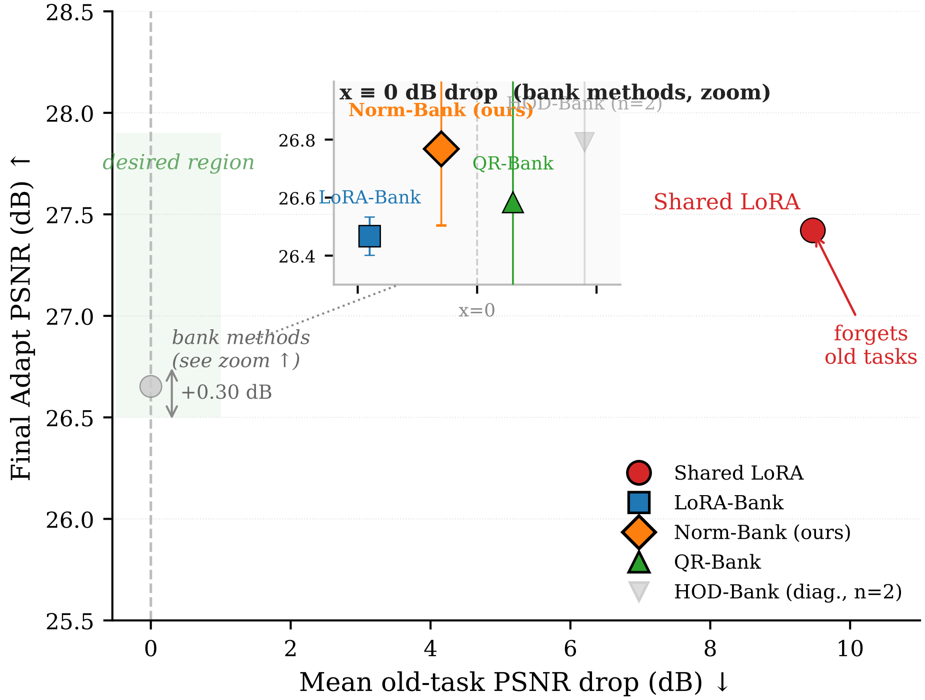

Fig. 2: Initial 20-image retention–plasticity evaluation (PromptIR, 10 ep/task, 3 seeds, mean ± std). Shared LoRA reaches the highest Adapt PSNR (27.42 dB) but forgets old tasks by 9.47 dB; all bank methods lie on the 0 dB old-task-drop line. Norm-Bank exceeds LoRA-Bank by +0.30 dB on this small slice, while the 200-image control in Table 3 reduces the corresponding five-seed gap to +0.016 dB. HOD-Bank is diagnostic only ($n = 2$, seed 42 missing).

Routing-mode effect. In the initial evaluation, the Norm-Bank cumulative-minus-task-only deltas are +0.723, −0.271, +0.205, +0.314, +0.183, a positive direction in 4 of 5 seeds and a mean gain of +0.231 dB. The corresponding LoRA-Bank gain is +0.052 dB with no consistent direction. On the 200-image evaluation, the two shared-seed cumulative-minus-task-only deltas are +0.631 and +0.648 dB, for a paired estimate of +0.640 dB. Together with the six reduced-sample cells in Section 5.4 (+0.541 to +0.881 dB), this strengthens the conclusion that cumulative routing is the dominant plasticity lever.

Initialization effect. The initial 20-image evaluation gives a cumulative-routing Norm-vs.-LoRA gap of +0.215 dB (4/5 seeds positive), consistent in direction with the three-seed screen in Table 1. Both methods drop on the harder 200-image evaluation, however: Norm-Bank reaches 25.882±0.225 dB and LoRA-Bank reaches 25.866 ± 0.247 dB. Their gap contracts to +0.016 dB, a 93% reduction. Although 4/5 seeds retain the direction, the mean is comparable to

Table 2: Initial 20-image full-sample PromptIR 2×2 evaluation. Adapt PSNR after task 2; seeds 42–46; 10 epochs per task. Every cell has mean old-task drop 0.000 dB.

Initialization	Routing	s42	s43	s44	s45	s46	Mean ± Std
Norm-Bank	task-only	26.173	26.961	26.280	26.602	26.498	26.503 ± 0.275
Norm-Bank	cumulative	26.896	26.690	26.485	26.916	26.681	26.734 ± 0.159
LoRA-Bank	task-only	26.666	26.658	26.469	26.709	25.834	26.467 ± 0.327
LoRA-Bank	cumulative	26.045	26.527	26.974	26.587	26.462	26.519 ± 0.297

Table 3: Evaluation-size control using the same trained checkpoints. The expanded evaluation covers 200 images and 14 checkpoints across the four Norm/LoRA × cumulative/task-only cells; no retraining is performed. The routing estimate uses the two seeds shared by cumulative and task-only evaluation.

Quantity	20 images	200 images
Routed Rain PSNR	36.81 ± 0.001	36.806 ± 0.001
Routed Snow PSNR	29.62 ± 0.10	29.62 ± 0.05
Old-task drop (dB)	0.000	0.0000
Norm-Bank cumulative	26.734 ± 0.159	25.882 ± 0.225
LoRA-Bank cumulative	26.519 ± 0.297	25.866 ± 0.247
Norm − LoRA (dB)	+0.215	+0.016
Cumulative − task-only (dB)	+0.231	+0.640

its standard error, so the expanded evaluation does not support a meaningful final-PSNR advantage from initialization alone.

Retention and evaluation-size effect. Across all four method–routing cells, routed rain and snow PSNR are identical at four-decimal precision before and after later tasks, giving 0.0000 dB old-task drop on 200 images. Final-task PSNR decreases by 0.852 dB for Norm-Bank and 0.653 dB for LoRA-Bank relative to the initial slice, indicating that the 20-image subset was systematically easier rather than revealing a method-specific shift.

Interpretation. The larger test set separates the two axes cleanly. Routing mode has a robust end-to-end effect, while row normalization has a robust birth-step mechanism but only a direction-consistent, near-zero final-PSNR difference on 200 images. We therefore do not treat the original +0.215 dB estimate as a headline effect.

5.4 Reduced-Sample Routing Sweep

Table 4 isolates routing mode across training budgets. Cumulative routing improves final Adapt PSNR over task-only routing in all six reduced-sample cells, by +0.541 to +0.881 dB. The effect shrinks as the per-task budget grows but stays positive for both LoRA-Bank and Norm-Bank, making routing mode the strongest plasticity lever we observe. The 200-image full-sample control in Table 3 independently gives a +0.640 dB paired estimate on its two shared seeds.

Table 4: Routing-mode control. Δ is cumulative minus task-only on final Adapt PSNR (dB), 3-seed mean ± sample std. Reduced-sample budget: Rain100L=100, Snow100K-L=1000, Adapt=400.

Budget	LoRA-Bank Δ	Norm-Bank Δ
1 epoch	+0.717 ± 0.107	**+0.881 ± 0.362**
3 epochs	+0.618 ± 0.126	**+0.629 ± 0.089**
10 epochs (reduced)	+0.541 ± 0.390	**+0.579 ± 0.557**

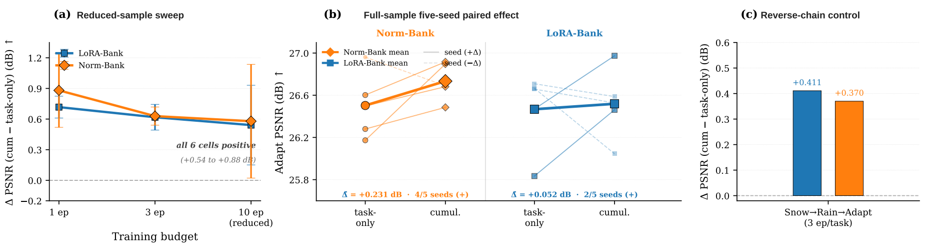

Fig. 3: Routing-mode effect (cumulative − task-only on final Adapt PSNR). **(a)** Reduced-sample forward-chain sweep: cumulative routing gains +0.54 to +0.88 dB across all six cells. **(b)** The initial 20-image full-sample evaluation gives +0.231 dB for Norm-Bank and +0.052 dB for LoRA-Bank; the 200-image control increases the shared-seed paired estimate to +0.640 dB. **(c)** The reverse-chain control gives +0.411 dB for LoRA-Bank and +0.370 dB for Norm-Bank.

5.5 Birth-Step Mechanism Diagnostic

Table 5 reports the single-batch diagnostic. LoRA-Bank shows a positive correlation between A row norm and first-step effective update magnitude ($\rho = +0.179$, 78% of layers positive). Norm-Bank cuts the mean correlation by 9.3× to +0.019; QR-Bank reaches −0.008, yet the end-to-end PSNR results show no consistent QR advantage over Norm-Bank, which places the active factor at row-scale calibration rather than strict row orthonormality. The Restormer diagnostic (seed 42, 84 LoRA-target layers) follows the same pattern: the active adapter moves from $\rho = +0.184$ under Kaiming initialization to −0.034 under row normalization (and −0.031 under QR), so the mechanism is backbone-transferable even where the final PSNR gain is setting-dependent.

5.6 Learned-Router Stress Test

The bank tables isolate the adapter mechanism with oracle routes. Table 6 evaluates a learned router on the same three degradation families. A frozen-feature logistic-regression router reaches 84.0% routing accuracy, well above the centroid router (66.7%), random (28.7%), and majority (33.3%) baselines. Snow is the most separable family (88%), and the main confusion is between rain and raindrop, which share texture structure between rain streaks and adherent lens raindrops [19].

Table 5: Birth-step scale-leakage diagnostic. PromptIR rows aggregate per-layer Pearson correlations over 164 LoRA-target layers and three seeds. Restormer rows are a seed-42 control over 84 target layers.

Backbone	Method	ρ mean	Row scale	Interpretation
PromptIR	LoRA-Bank	$+0.179 \pm 0.009$	Kaiming	Leakage (78% positive)
PromptIR	**Norm-Bank**	$\mathbf{+0.019} \pm \mathbf{0.015}$	Unit norm	9.3× reduction
PromptIR	QR-Bank	-0.008 ± 0.017	Orthonormal	No PSNR gain over Norm
Restormer	LoRA-Bank	$+0.184$	Kaiming	Leakage present
Restormer	**Norm-Bank**	$\mathbf{-0.034}$	Unit norm	Leakage removed

Table 6: Frozen-feature route prediction. The router is fit on 50 images per family and evaluated on 50 held-out images per family.

Policy / Router	Accuracy	Rain	Snow	Raindrop
Oracle	1.000	1.000	1.000	1.000
Centroid	0.667	0.72	0.70	0.58
Logistic regression	**0.840**	**0.78**	**0.88**	**0.86**
Random	0.287	–	–	–
Majority	0.333	1.00	0.00	0.00

Table 7 reports auto-routed PSNR under the logistic-regression router. The oracle-to-auto gap is 1.413 dB for Norm-Bank and 1.382 dB for LoRA-Bank under cumulative routing, and the auto-routed means are effectively tied (30.060 vs. 30.120 dB, with 2 of 5 seeds favoring Norm-Bank). Bank design governs oracle-route plasticity, while frozen task-isolated routing gives route-level retention. The latter is the structural property: completed adapters are frozen and historical routes are reused unchanged, so it holds regardless of initialization or routing mode. Route prediction, in contrast, governs blind-deployment quality: the 1.38–1.41 dB oracle-to-auto gap exceeds the bank-design difference, making routing the primary deployment bottleneck. Row normalization does not increase sensitivity to this penalty. The same lightweight router's accuracy under task-scale expansion ($T = 1, \ldots, 7$) is reported in Supplementary Appendix E.

5.7 Backbone and Chain Controls

The reverse PromptIR chain (snow → rain → adapt, 3 epochs/task, 3 seeds) reproduces both findings: bank methods hold old-task drop at 0.000 dB, and cumulative routing keeps its advantage over task-only routing by +0.411 dB for LoRA-Bank and +0.370 dB for Norm-Bank. The Restormer control confirms that route-level non-interference is not specific to PromptIR: shared LoRA forgets old tasks by 8.21 dB mean, while all bank methods preserve old-task

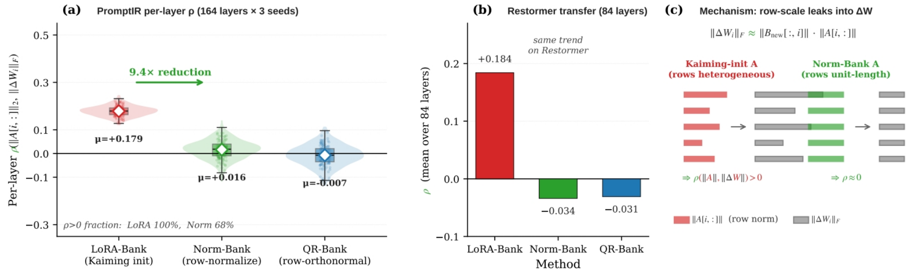

Fig. 4: Birth-step scale-leakage diagnostic. Per-layer Pearson correlation ρ between the Kaiming A row norm $\|A_t[i,:]\|_2$ and the first-step effective update $\|\Delta W_{t,i}\|_F = (\alpha/r)\,\|B_{\text{new}}[\cdot,i]\|_2\,\|A_t[i,:]\|_2$ across $r=8$ ranks, 164 LoRA-target layers, 3 seeds. LoRA-Bank has $\rho=+0.179$ (78% of layers positive). Row normalization at birth (Norm-Bank) reduces ρ to $+0.019$ ($9.3\times$ reduction); QR-Bank reaches -0.008 ($23.7\times$). Norm-Bank and QR-Bank set the active adapter's row norms to unity up to numerical precision, confirming that the active factor is row-scale calibration, not orthonormality.

routed PSNR at 0.000 dB across three seeds [28]. On Restormer the final-task gap among bank methods is small (Norm-Bank 27.819, LoRA-Bank 27.815, QR-Bank 28.009 dB), so we read Restormer as evidence for route preservation and mechanism transfer rather than for a universal Norm-Bank PSNR advantage.

5.8 Additional Controls

Rank 8 reaches the observed capacity plateau, adapter storage grows linearly at about 1.9 MB per PromptIR task in fp16, and all primary methods pass drift, reload, and NaN checks. Detailed rank, storage, and engineering results appear in Supplementary Appendix B.

6 Discussion

Effects. Frozen task routes govern non-interference, cumulative routing governs plasticity, and row-normalized birth governs first-step scale. The 200-image evaluation sharpens this separation: old-task drop remains 0.0000 dB and the shared-seed routing gain is +0.640 dB, whereas the cumulative Norm-vs.-LoRA gap contracts to +0.016 dB. Row normalization therefore has a reproducible mechanism but no established practical PSNR advantage.

Table 7: Logistic-regression auto-router PSNR. Cumulative rows use five seeds; task-only rows are two-seed route-sensitivity checks. Oracle and auto PSNR are measured on the router test split and compared only within this table; they are not directly comparable with the Adapt PSNR in Tables 1–4, which uses a different evaluation set.

Method	Routing	Oracle PSNR	Auto PSNR	Oracle–auto gap	n
Norm-Bank	cumulative	31.474 ± 0.106	30.060 ± 0.143	1.413 ± 0.130	5
LoRA-Bank	cumulative	31.502 ± 0.078	30.120 ± 0.105	1.382 ± 0.088	5
Norm-Bank	task-only	31.484 ± 0.102	29.848 ± 0.151	1.637 ± 0.049	2
LoRA-Bank	task-only	31.415 ± 0.075	29.868 ± 0.070	1.547 ± 0.005	2

Deployment bottleneck. The learned router reaches 84.0% accuracy, yet its 1.38–1.41 dB oracle-to-auto gap exceeds the bank-design difference. Blind quality is thus governed mainly by route prediction, while the frozen bank governs oracle-route retention. Stronger degradation classifiers are the most direct deployment improvement.

Relation to orthogonal methods. Orthogonal continual-learning methods constrain update directions [10, 21, 24]; Norm-Bank instead calibrates row scale once at birth. QR-Bank and HOD-Bank show no benefit from stricter initialization geometry in our setting, although combining scale calibration with pretrained subspaces or adaptive budgets remains promising.

7 Conclusion

Task-routed LoRA banks separate continual restoration into three effects. Frozen historical routes preserve old-task PSNR at 0.0000 dB drop; cumulative routing supplies plasticity, with a +0.640 dB expanded-evaluation estimate; and row-normalized birth removes first-step scale leakage (ρ : +0.179 $\rightarrow$ +0.019) but yields only a +0.016 dB final-PSNR gap. The learned router's 1.38–1.41 dB oracle-to-auto penalty is substantially larger, making route prediction the main deployment bottleneck.

References

1. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: ImageNet: A large-scale hierarchical image database. In: IEEEConf.Comput.Vis.PatternRecog. pp. 248–255 (2009) 7
2. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An image is worth 16×16 words: Transformers for image recognition at scale. In: Int.Conf.Learn.Represent. (2021) 3
3. He, J., Duan, Z., Zhu, F.: CL-LoRA: Continual low-rank adaptation for rehearsal-free class-incremental learning. In: IEEEConf.Comput.Vis.PatternRecog. pp. 30534–30544 (2025) 3

4. He, K., Zhang, X., Ren, S., Sun, J.: Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In: Int.Conf.Comput.Vis. pp. 1026–1034 (2015) 2, 4, 7
5. Hsu, Y.C., Liu, Y.C., Ramasamy, A., Kira, Z.: Re-evaluating continual learning scenarios: A categorization and case for strong baselines. In: NeurIPS Continual Learning Workshop (2018) 4
6. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: LoRA: Low-rank adaptation of large language models. In: Int.Conf.Learn.Represent. (2022) 1, 3, 4, 7
7. Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., Hadsell, R.: Overcoming catastrophic forgetting in neural networks. Proc.Natl.Acad.Sci.U.S.A. **114**(13), 3521–3526 (2017) 4
8. Li, R., Tan, R.T., Cheong, L.F.: All in one bad weather removal using architectural search. In: IEEEConf.Comput.Vis.PatternRecog. pp. 3175–3185 (2020) 1, 5
9. Li, Z., Hoiem, D.: Learning without forgetting. IEEETrans.PatternAnal.Mach.Intell. **40**(12), 2935–2947 (2018) 4
10. Liang, Y.S., Li, W.J.: InfLoRA: Interference-free low-rank adaptation for continual learning. In: IEEEConf.Comput.Vis.PatternRecog. pp. 23638–23647 (2024) 3, 14
11. Liu, S.Y., Wang, C.Y., Yin, H., Molchanov, P., Wang, Y.C.F., Cheng, K.T., Chen, M.H.: DoRA: Weight-decomposed low-rank adaptation. In: Int.Conf.Mach.Learn. (2024) 3
12. Liu, Y.F., Jaw, D.W., Huang, S.C., Hwang, J.N.: DesnowNet: Context-aware deep network for snow removal. IEEETrans.ImageProcess. **27**(6), 3064–3073 (2018) 5
13. Lopez-Paz, D., Ranzato, M.: Gradient episodic memory for continual learning. In: Adv.NeuralInform.Process.Syst. (2017) 4
14. Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: Int.Conf.Learn.Represent. (2019) 7
15. Mallya, A., Lazebnik, S.: PackNet: Adding multiple tasks to a single network by iterative pruning. In: IEEEConf.Comput.Vis.PatternRecog. (2018) 3, 4
16. Meng, F., Wang, Z., Zhang, M.: PiSSA: Principal singular values and singular vectors adaptation of large language models. In: Adv.NeuralInform.Process.Syst. (2024) 3
17. Parisi, G.I., Kemker, R., Part, J.L., Kanan, C., Wermter, S.: Continual lifelong learning with neural networks: A review. Neural Netw. **113**, 54–71 (2019) 2
18. Potlapalli, V., Zamir, S.W., Khan, S., Khan, F.S.: PromptIR: Prompting for all-in-one image restoration. In: Adv.NeuralInform.Process.Syst. (2023) 1, 3, 7
19. Qian, R., Tan, R.T., Yang, W., Su, J., Liu, J.: Attentive generative adversarial network for raindrop removal. In: IEEEConf.Comput.Vis.PatternRecog. pp. 2482–2491 (2018) 5, 11
20. Rusu, A.A., Rabinowitz, N.C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., Hadsell, R.: Progressive neural networks. In: arXiv preprint arXiv:1606.04671 (2016) 4
21. Saha, G., Garg, I., Roy, K.: Gradient projection memory for continual learning. In: Int.Conf.Learn.Represent. (2021) 3, 14
22. Valanarasu, J.M.J., Yasarla, R., Patel, V.M.: TransWeather: Transformer-based restoration of images degraded by adverse weather conditions. In: IEEEConf.Comput.Vis.PatternRecog. pp. 2353–2362 (2022) 3
23. Wan, Z., Du, W., Li, L., Pan, M., Qin, X.: OA-Adapter: Adaptive budget allocation for orthogonal-subspace adapter tuning in LLMs continual learning. arXiv preprint arXiv:2505.22358 (2025) 3

24. Wang, X., Chen, T., Ge, Q., Xia, H., Bao, R., Zheng, R., Zhang, Q., Gui, T., Huang, X.: Orthogonal subspace learning for language model continual learning. In: Find.Assoc.Comput.Linguist.:EMNLP. pp. 10658–10671 (2023) 3, 14
25. Wang, Z., Bovik, A.C., Sheikh, H.R., Simoncelli, E.P.: Image quality assessment: From error visibility to structural similarity. IEEETrans.ImageProcess. **13**(4), 600–612 (2004) 8
26. Wu, C., Wang, P., Zheng, Z.: Re-examine all-in-one image restoration: A catastrophic forgetting perspective. PatternRecognit.Lett. **197**, 161–167 (2025). https://doi.org/10.1016/j.patrec.2025.07.004 1, 3
27. Yang, W., Tan, R.T., Feng, J., Liu, J., Guo, Z., Yan, S.: Deep joint rain detection and removal from a single image. In: IEEEConf.Comput.Vis.PatternRecog. pp. 1357–1366 (2017) 5
28. Zamir, S.W., Arora, A., Khan, S., Hayat, M., Khan, F.S., Yang, M.H.: Restormer: Efficient transformer for high-resolution image restoration. In: IEEEConf.Comput.Vis.PatternRecog. pp. 5728–5739 (2022) 7, 13
29. Zhang, Q., Chen, M., Bukharin, A., Karampatziakis, N., He, P., Cheng, Y., Chen, W., Zhao, T.: AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In: Int.Conf.Learn.Represent. (2023) 3